

Project – Development of a distributed application integrating Bicing, Telegram, IP Geolocation, Air Quality Index and Open Gateway services and deploying it in AWS

In this project, we have implemented a system that sends two types of messages via Telegram to users: one with the free slots at the Bicing stations they have previously subscribed to, and the other with the Air Quality Index (AQI) of their location, according to their IP address.

1. REST API Documentation

The main service, Bicing Free Slots/Air Quality Notifier (BAN), uses a REST API that gives access to Bicing data, user subscriptions, Telegram notifications, air quality monitoring and logging functionalities.

Our API has a layered architecture where the different resources handle HTTP requests and delegate the logic to the corresponding services. All responses are returned in JSON format.

1.1. Stations Service

GET /stations

The GET /stations endpoint, implemented in StationResource, retrieves the list of Bicing stations using CacheService, which decides if it returns the data in the cache or if it fetches fresh data from BicingService.

When we request this endpoint, we get a List<Station> object as response.

```
{
  "is_charging_station": true,
  "last_reported": 1772567230,
  "num_bikes_available": 3,
  "num_docks_available": 42,
  "station_id": 1,
  "status": "IN_SERVICE"
},
```

Figure 1: Output of a station.

1.2. Clients Service

The client service is implemented in ClientResource and allows clients the registration of new users and the retrieval of existing subscriptions.

GET /clients

The GET /clients endpoint returns all the registered clients in ClientRepository. When we request this endpoint, we get a Map<String, Client> as response.

```
[
  {
    "chat_id": 5578593207,
    "phone": "+34606918985",
    "stationsIDs": [
      1,
      2,
      3
    ],
    "telegramToken": "8375476740:AAF16doZ1TCzUBtSSuV4u6xTOK80d6YUPN4"
  }
]
```

Figure 2: Output of a client.

POST /clients

The POST /clients endpoint is the one responsible for registering new clients. Before storing the new client, the endpoint invokes AgeVerificationService to verify that the provided phone number belongs to an adult using the Open Gateway API.

If the verification succeeds, and the user is an adult, the new client is stored in ClientRepository (singleton).

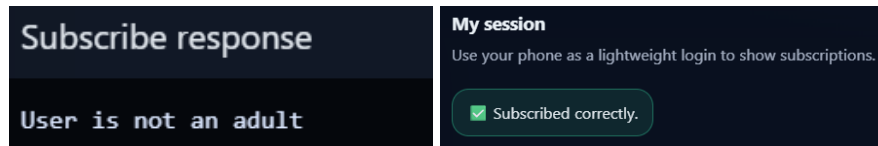


Figure 3: Responses when a user is not an adult and when the subscription is successful.

1.3. Notifier Service

The notifier service is implemented in NotifierResource and NotifierService and allows sending notifications to the registered users via Telegram.

POST /notify/slots

The POST /notify/slots endpoint is in charge of sending notifications with the availability of bikes in the stations subscribed by the client.

When requested, retrieves the client from ClientRepository and obtains the station data via CacheService. Then, it filters the stations based on the user's subscriptions and sends a Telegram message through TelegramService. The response is represented by NotifierResponse.

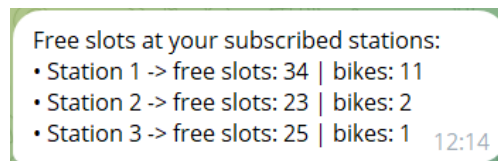


Figure 4: Telegram notification showing the availability of bikes in subscribed stations.

POST /notify/airquality

The POST /notify/airquality endpoint is the one that sends the notification with the air quality based on the client's IP address.

When requested, it determines the city from the provided IP address using IpGeoService, retrieves the AQI value with AqiService and creates an AirQuality object. Then the AQI value is translated to a qualitative level and, via TelegramService, a notification is sent to the user.

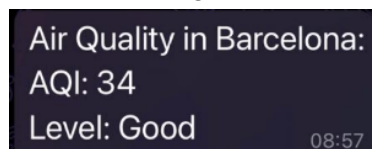


Figure 5: Telegram notification showing the air quality information based on the user's IP address.

1.4. Logs Service

The log service, implemented in LogsResource, and with the endpoint GET /logs, returns the last lines of the application logs to facilitate debugging and demonstration.

```

2026-03-04 17:58:00 DEBUG ClientRepository:44 - CLIENT GET phone=+34606918985 found=true
2026-03-04 17:58:00 INFO IpGeoService:53 - IP GEO ok ip=95.23.52.125 city=Barcelona
2026-03-04 17:58:02 WARN NotifierService:146 - NOTIFY_AIR_ERROR reason=aqi_not_found phone=+34606918985 city=Barcelona
2026-03-04 17:58:02 INFO RequestLoggingFilter:41 - HTTP_OUT GET /api/notifier/air/+34606918985/95.23.52.125 -> 400 (1411 ms)
2026-03-04 18:00:34 INFO RequestLoggingFilter:29 - HTTP_IN GET /api/logs
2026-03-04 18:00:34 INFO RequestLoggingFilter:41 - HTTP_OUT GET /api/logs -> 200 (2 ms)
2026-03-04 18:00:36 INFO RequestLoggingFilter:29 - HTTP_IN GET /api/logs

```

Figure 6: Response of the GET /logs endpoint showing recent application log entries.

2. Implementation

2.1. Architecture Overview

The BAN system has the following layered architecture and every layer has its own probability:

- **Filter:** RequestLoggingFilter
This filter intercepts incoming HTTP requests and logs relevant information for monitoring and debugging purposes.
- **REST Layer:** ClientResource, LogsResource, NotifierResource, NotifierResource, StationResource
This layer is responsible for handling HTTP requests and responses. Each resource defines the available endpoints and delegates the business logic to the corresponding service classes.
- **Service Layer:** AgeVerificationService, AqiService, BicingService, CacheService, IpGeoService, NotifierService, TelegramService
This layer contains the core business logic of the application. It manages the interactions between internal components and external API's, like Bicing, Open Gateway, AQI services, IP geolocation services and Telegram.
- **Repository Layer:** ClientRepository
This layer stores the registered clients in memory.
- **Domain Model:** AirQuality, Client, Data, Message, NotifierResponse, Station, Stations
These classes represent the main entries of the system and are used to model the data and map external API responses into internal objects.
- **Util:** Constants, HttpClientUtil
These classes provide reusable utility functionality, such as configuration values and centralize HTTP communication logic.

2.2. Cache Mechanism

The cache is implemented in CacheService and stores a list of cached stations and the time of the last update.

When the getStationsCached() method is invoked, if the cache is valid, cached data is returned; otherwise, BicingService is called to refresh station data. This reduces external API calls and improves performance.

2.3. Age Verification (Open Gateway)

Age verification is implemented in AgeVerificationService. When a new client registers, the service sends the phone number to the Open Gateway API and, if the user is an adult, it stores the client in ClientRepository.

The Telegram integration is managed by `TelegramService` which builds a `Message` object, performs an HTTP POST request to the Telegram Bot API, and returns the response as a `String`.

2.5. Air Quality Flow

NotifierService obtains the city from the provided IP and retrieves the corresponding AQI value. It then creates an AirQuality object and sends the formatted notification to the user via TelegramService.

Logging is implemented across the different services to track cache hits and refreshes, external API calls, client registration attempts and Telegram message delivery. Logs are essential to validate the cache effectiveness and notification execution flow.

```

classDiagram
    class ClientResource {
        +getClients()
        +subscribe(client)
    }
    class LoggerResource {
        +getLogs(lines)
    }
    class NotifierResource {
        +notifySlots(phone)
        +notifyAirQuality(phone, ip)
    }
    class StationResource {
        +getAllStations()
    }
    class AgeVerificationService {
        +isAdult(phone): boolean
    }
    class NotifierService {
        +notifySlots(phone): NotifierResponse
        +notifyAirQuality(phone, ip): NotifierResponse
    }
    class NotifierResponse {
        -status: String
        -message: String
    }
    class CacheService {
        -cachedStations: List<Stations>
        -cache_ts: long
        +getStationsCached(): List<Station>
    }
    class BicingService {
        +getStations(): Data
    }
    class Stations {
        -stations: List<Station>
    }
    class Data {
        -data: Stations
    }
    class Station {
        -station_id: int
        -num_bikes_available: int
        -num_docks_available: int
        -status: String
    }
    class ClientRepository {
        <<Singleton>>
        +clients: Map<String, Client>
        +getInstance()
        +addClient(client)
        +getClientByPhone(phone)
        +getAllClients()
    }
    class Client {
        -phone: String
        -telegramToken: String
        -chat_id: Long
        -stationsIds: List<Integer>
    }
    class AqiService {
        +getAqByCity(city): Integer
    }
    class IpGeoService {
        +getCityByIp(ip): String
    }
    class TelegramService {
        +sendMessage(token, message): String
    }
    class Message {
        -chat_id: long
        -text: String
    }
    class AirQuality {
        -city: String
        -aqi: int
        -level: String
    }

    ClientResource --> ClientRepository
    ClientResource --> Client
    LoggerResource --> NotifierService
    NotifierResource --> NotifierService
    NotifierResource --> NotifierResponse
    StationResource --> CacheService
    AgeVerificationService --> NotifierService
    NotifierService --> NotifierResponse
    NotifierService --> CacheService
    NotifierService --> ClientRepository
    NotifierService --> Client
    NotifierService --> AqiService
    NotifierService --> IpGeoService
    NotifierService --> TelegramService
    NotifierService --> AirQuality
    NotifierService --> Station
    CacheService --> BicingService
    CacheService --> Station
    BicingService --> Stations
    Stations --> Data
    Data --> Station
  
```